

ISM3232 — Week 5 Lab

Variables, Data Types & Operators — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 05 · Unit 2 · Python Foundations

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Timing Plan (75 minutes)	3
6	Segment-by-Segment Walkthrough	5
6.1	Intro (0:00–0:03)	5
6.2	Part 1 — Setup and All Four Data Types (0:03–0:17, 14 min)	5
6.3	Part 2 — Operators and f-strings (0:17–0:30, 13 min)	8
6.4	Part 3 — User Input with Type Conversion (0:30–0:43, 13 min)	10
6.5	Part 4 — pytest (0:43–1:03, 20 min)	12
6.6	Part 5 — Submission Ritual and Push (1:03–1:11, 8 min)	16
6.7	Stretch — Formatted Report Block (as time allows)	18
7	Wrap-Up (last ~4 minutes)	19
8	Appendix A — Full Answer Key (week5_lab.py + tests/test_week5.py)	19
9	Appendix B — Extra Practice (only if the class finishes early)	20

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 5 Lab — Variables, Data Types & Operators
Unit	Unit 2 · Python Foundations
Class length	Full class period (75 minutes)
Format	Live code-along, ending with the full Week 4 submission ritual
Prerequisites	Weeks 1–4: verified environment, venv/.gitignore/.zshrc, search tools, pytest, the ritual, a working GitHub remote (marked Midterm-Eligible)
Student-facing lab page	Week 5 In-Class Lab — Module 4, “Variables, Data Types, Operators, and f-strings”
Parts covered	Part 1 (setup + four types) – Part 5 (ritual + push) + Stretch (formatted report)
Submission	Git commit (message must include “lab 5”) + Canvas URL, completion credit

Unit 1 was entirely about the *environment*; this is the first lab where that environment finally runs real Python content. Everything from Weeks 1–4 — the venv, the ritual, pytest — exists specifically to support what starts today. **Read this guide’s Part 4 walkthrough carefully before class:** the lab page’s own provided `test_week5.py` file, run exactly as written, produces **three failing tests out of five**, not the “all five must pass” the lab page states. This isn’t a typo to route around quietly — it’s two genuinely excellent, real Python gotchas (floating-point equality and chained comparison syntax) hiding inside a normal-looking test file, and this guide turns that into the lab’s best teaching moment rather than an embarrassing surprise mid-class.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Create variables of all four core types (`str`, `int`, `float`, `bool`) and confirm each with `type()`.
2. Compute derived business values (`subtotal`, `tax`, `total`) using arithmetic operators, and format them with f-string format specs.
3. Convert `input()`’s string return value with `int()/float()` before doing arithmetic, and explain the resulting `TypeError` when conversion is skipped.
4. Write and run `pytest` assertions — and recognize two specific, real Python traps: comparing floats with `==`, and chaining a comparison directly against `is True/is False`.

5. Execute the full pre-submission ritual from Week 4 on genuinely new content, without prompting.

3 Before Class — Setup Checklist

- ☐ **Critical: run the lab page’s own provided `test_week5.py` yourself, exactly as written, before class.** As distributed, `test_over_limit_true`, `test_over_limit_false`, and `test_tax_at_seven_percent` all **fail** — not because of any student error, but because of two genuine Python subtleties baked into the provided code (detailed fully in Part 4 below). Decide now whether you want to present this as a live “even provided code can be wrong — let’s diagnose why” moment (recommended — it’s a genuinely excellent lesson and fits this course’s debugging culture) or simply hand out the corrected version from Appendix A without dwelling on it. Either way, **do not be caught off guard live** by red tests you weren’t expecting.
- ☐ Rehearse your own version of the Part 1–3 script once, choosing a business domain, so your live-coded numbers are clean and easy to read from the back of the room (this guide uses a laptop purchase-request scenario throughout, matching the lab page’s own template).
- ☐ Confirm every student’s Week 4 GitHub remote still works — a quick `git status` in `module04_programming/` (a **different** folder than Week 4’s `module02_zsh/`, note explicitly) will need its *own* fresh `venv` and its *own* remote connection, not a reuse of Week 4’s.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, from the established Weeks 1–4 workflow
- Students: a **new** project folder, `~/ism3232/module04_programming/`, with its own fresh `venv` (Part 1 sets this up from scratch)

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:03	Welcome: “the environment work pays off starting now”	3
0:03–0:17	Part 1 — Setup and all four data types	14
0:17–0:30	Part 2 — Operators and f-strings	13
0:30–0:43	Part 3 — User input with type conversion	13
0:43–1:03	Part 4 — pytest (including the two real bugs)	20

Time	Segment	Minutes
1:03–1:11	Part 5 — Submission ritual and push	8
1:11–1:15	Stretch preview + wrap-up	4

Part 4 is deliberately given the most time of any single part in this lab — diagnosing the two real bugs properly, rather than rushing past them, is worth the extra minutes; if your section needs to compress somewhere, Part 5’s ritual is the fastest to move through quickly since it’s pure repetition of Week 4’s already-established sequence.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:03)

Say to the class:

“Four weeks of environment work — venvs, aliases, Git, testing infrastructure — all so that today, and every lab from here forward, you can focus entirely on the Python itself. Today: all four core data types, the operators that combine them, and your first genuinely substantial pytest file. Fair warning up front: not everything in today’s lab is going to work exactly as expected the first time — and that’s deliberate. Two of today’s five tests are going to fail when we first run them, and figuring out why is one of the best lessons in this entire course.”

6.2 Part 1 — Setup and All Four Data Types (0:03–0:17, 14 min)

Teaching goal: A fresh project setup (new venv, new `.gitignore`) in a new folder, and the four core Python types — `str`, `int`, `float`, `bool` — confirmed with `type()`.

Say to the class:

“New module, new folder, new venv — this is genuinely a fresh project, not a continuation of Week 4’s. Set it up the same way, every time, from muscle memory now.”

Live-code this:

```
cd ~/ism3232/module04_programming
python3 -m venv .venv && source .venv/bin/activate
pip install pytest ruff && pip freeze > requirements.txt
echo '.venv/' > .gitignore && echo '__pycache__/' >> .gitignore
touch week5_lab.py && code week5_lab.py
```

Line-by-line explanation:

- Every line here is Week 3’s exact setup sequence, chained with `&&` to run efficiently — say explicitly: **this five-line block is worth memorizing as “how every new ISM3232 project starts,”** since it recurs, with only the folder name changing, at the start of most remaining modules this semester.
- `&&` between commands — new usage today, though the underlying idea (Week 3’s `mkcd` function) already introduced it: each command only runs if the one before it succeeded — a small safety net against, say, trying to activate a venv that failed to create.

Write the script — choose a business domain freely; this guide uses a purchase request:

```
# week5_lab.py
# Author: [Your Name]
# Business domain: [describe your scenario]

product_name = 'Laptop'
status       = 'Pending'
quantity     = 3
unit_price   = 450.00
is_over_limit = unit_price * quantity > 1000

print(type(product_name), type(quantity), type(unit_price), type(is_over_limit))
```

Line-by-line explanation:

- `product_name = 'Laptop'` — a `str`.
- `status = 'Pending'` — another `str` — worth noting two string variables exist here for different purposes (one is data to display, the other, `status`, hints at future state-tracking logic not yet built) — a small preview that not every variable needs to be immediately used in a calculation to be meaningful.
- `quantity = 3` — an `int`.
- `unit_price = 450.00` — a `float` (the decimal point is what makes it one, regardless of whether the value happens to be a whole number — the same rule from every prior Python course's Module 01/03 equivalent).
- `is_over_limit = unit_price * quantity > 1000` — a `bool`, and worth reading the **order of operations** explicitly: `*` binds tighter than `>`, so this computes `(unit_price * quantity) > 1000` — `450.00 * 3 = 1350.0`, and `1350.0 > 1000` is `True`. Ask the room to confirm this is what they'd expect *before* running it, not after.
- `print(type(...), type(...), type(...), type(...))` — passing four separate `type()` calls as four separate arguments to one `print()` — say explicitly this is new relative to prior `print-one-type-per-line` patterns: `print()` accepts any number of comma-separated arguments and prints them all on one line, space-separated by default.

Run it. Verified output:

```
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
```

Common student mistakes to watch for:

- Running the setup block from the wrong location, or forgetting source `.venv/bin/activate` before `pip install` — Week 3's exact failure modes, worth a quick `which python3` check if `pip install` behaves unexpectedly.
- Writing `unit_price = 450` (no decimal) out of habit — `type()` would then report `int`, not `float`, silently changing what this exercise is meant to demonstrate; worth a quick visual double-check across the room.

Check for understanding: “If I asked for `is_over_limit` to check whether the order needs approval at a \$2,000 threshold instead of \$1,000, what’s the one-character change?” (Change `1000` to `2000` — a good, quick confirmation that students can read and modify the comparison, not just recognize it.)

6.3 Part 2 — Operators and f-strings (0:17–0:30, 13 min)

Teaching goal: Derived business calculations (subtotal, tax, total) and a fully formatted, multi-line business summary using f-strings.

Say to the class:

“Three more calculated values, then a clean, formatted printout — this is what a real business script’s output should look like, not just raw numbers.”

Live-code this, added to the same file:

```
subtotal      = unit_price * quantity
tax           = subtotal * 0.07
total         = subtotal + tax
requires_approval = total > 1000

print('=== Purchase Request Summary ===')
print(f'Product: {product_name}')
print(f'Qty:      {quantity}')
print(f'Subtotal: ${subtotal:.2f}')
print(f'Tax:      ${tax:.2f}')
print(f'Total:    ${total:.2f}')
print(f'Requires approval: {requires_approval}')
```

Line-by-line explanation:

- `subtotal = unit_price * quantity` — $450.00 * 3 = 1350.0$.
- `tax = subtotal * 0.07` — a 7% tax rate, applied to the subtotal — worth a brief flag here that this exact calculation is about to matter a great deal in Part 4: **`subtotal * 0.07` does not always produce a perfectly clean decimal in floating-point arithmetic**, even though the math looks simple on paper. Hold that thought; Part 4 makes it concrete.
- `total = subtotal + tax` — the final charged amount.
- `requires_approval = total > 1000` — a second, independently-computed boolean, distinct from Part 1’s `is_over_limit` (which used a *different* threshold check on the pre-tax amount) — worth pointing out these two booleans could, in principle, disagree for some inputs, since they’re checking genuinely different things.
- `:.2f` on every dollar amount — the currency format spec, consistent across all three lines, exactly the convention established in every prior module’s business-formatting work.
- `f'Requires approval: {requires_approval}'` — printing a boolean directly inside an f-string — displays as the literal word `True` or `False`, no special handling needed.

Run it. Verified output:

```
=== Purchase Request Summary ===
```

Product: Laptop
Qty: 3
Subtotal: \$1350.00
Tax: \$94.50
Total: \$1444.50
Requires approval: True

Common student mistakes to watch for:

- Computing tax from total instead of subtotal by a copy-paste slip — produces a compounding error (tax-on-tax) rather than a clean 7% of the pre-tax amount; a good “does this number look plausibly like 7%” sanity check if the room’s totals look off.
- Forgetting the `:.2f` on one of the three dollar lines — not an error, just an inconsistent-looking report; walk the room checking all three currency lines share the same format spec.

Check for understanding: “If subtotal were exactly \$1000.00, would `requires_approval` be True or False, given tax is added on top?” (True — \$1000.00 + 7% tax pushes the total to \$1070.00, over the \$1000 threshold, even though the subtotal alone sits exactly at the boundary; a good reminder that `requires_approval` checks the *post-tax* total specifically, not the subtotal.)

6.4 Part 3 — User Input with Type Conversion (0:30–0:43, 13 min)

Teaching goal: `input()` always returns a string — convert with `int()` before arithmetic, or hit a genuinely instructive `TypeError`.

Say to the class:

“One line of new user-facing input, and I want to show you the single most common Week 5 bug on purpose, before it happens to you by accident.”

Live-code this, added to the end of the file:

```
user_qty = int(input('Enter a new quantity: '))
new_total = unit_price * user_qty * 1.07
print(f'New total for {user_qty} units: ${new_total:.2f}')
print(f'Requires approval: {new_total > 1000}')
```

Line-by-line explanation:

- `int(input('Enter a new quantity: '))` — read inside-out: `input(...)` displays the prompt and returns whatever was typed, **as a string, always** — `int(...)` then converts that string to a genuine integer.
- `unit_price * user_qty * 1.07` — a slightly different (but equivalent) way of computing a taxed total than Part 2’s separate subtotal/tax/total variables: multiplying by 1.07 directly, in one step, applies the 7% tax inline. Worth a brief note that both styles are valid; this one is more compact, Part 2’s is more explicit about each intermediate value — a real style tradeoff, not a correctness difference.

Run it and enter a number, e.g. 5. Verified output:

```
Enter a new quantity: 5
New total for 5 units: $2407.50
Requires approval: True
```

Now, deliberately show the bug, exactly as the lab page itself frames it — “the most common Week 5 bug”:

```
unit_price * input('qty: ')
```

Run it and enter any number. It raises:

```
TypeError: can't multiply sequence by non-int of type 'float'
```

Explain precisely why, since the message reads oddly the first time: `input('qty: ')` returns a **string**, unconverted. `unit_price * <that string>` is Python attempting to multiply a float by a string — and Python’s error phrasing here comes from thinking about it from the string’s side: strings support repetition when multiplied by an *integer* (`'ab' * 3` gives `'ababab'`), so the error is

really saying “I tried to treat this like string-repetition, but the other side is a `float`, not a valid repeat count.” Say plainly: **the specific wording is less important than the takeaway — `input()` is always a string until you explicitly convert it, no matter what a user actually typed.**

Common student mistakes to watch for:

- Converting with `float()` instead of `int()` for a quantity — runs without error, but changes `user_qty`’s type, which would print as `5.0` instead of `5` in the f-string unless the format spec compensates; worth asking why a quantity specifically calls for `int()` rather than `float()` here (a fractional number of laptops doesn’t make business sense).
- Entering non-numeric text at the prompt (e.g., “five”) — raises `ValueError: invalid literal for int() with base 10: 'five'`, a different error from the multiplication `TypeError` shown above; worth a brief note that these are two distinct failure modes from two distinct causes (a missing conversion vs. a conversion given genuinely unconvertible text).

Check for understanding: “Why does the error say ‘can’t multiply sequence’ — what does Python mean by calling a string a ‘sequence’ here?” (A string is one specific kind of Python **sequence** — an ordered collection of characters — and sequences in general support multiplication-as-repetition by an integer; the error is using that general vocabulary, not something specific to strings alone. Not required depth for this course, but a good “why does this specific wording exist” answer for a curious student.)

6.5 Part 4 — pytest (0:43–1:03, 20 min)

Teaching goal: Write and run five pytest assertions — and diagnose two genuinely real, subtle Python bugs hiding in the provided test file, rather than just confirming green output.

Say to the class:

“Five tests. I’m going to type them exactly as given, run them, and I want you to watch closely — because not all five are going to pass, and the reason why is one of the most useful things you’ll learn all semester about how Python actually works underneath what looks like simple code.”

Live-code this:

```
mkdir -p tests && touch tests/__init__.py tests/test_week5.py
code tests/test_week5.py
```

Type the tests exactly as provided:

```
def test_tax_at_seven_percent():
    subtotal = 200.00
    tax = subtotal * 0.07
    assert tax == 14.0

def test_over_limit_true():
    assert 1500 > 1000 is True

def test_over_limit_false():
    assert 500 > 1000 is False

def test_type_of_string():
    name = 'ISM3232'
    assert type(name) == str

def test_type_conversion():
    s = '42'
    assert int(s) == 42
```

Run:

```
pytest -v
```

Actual verified output (not what the lab page’s “all five must pass” instruction implies):

```
tests/test_week5.py::test_tax_at_seven_percent FAILED
tests/test_week5.py::test_over_limit_true FAILED
tests/test_week5.py::test_over_limit_false FAILED
```

```
tests/test_week5.py::test_type_of_string PASSED
tests/test_week5.py::test_type_conversion PASSED
2 passed, 3 failed
```

Say explicitly, calmly, framing this as the actual point of the exercise rather than a problem to apologize for: “Three failures. Nobody typed anything wrong — this is exactly what the code as given does. Let’s diagnose each one, because both underlying causes are things you will run into again, for real, later in your career.”

Bug 1 — test_tax_at_seven_percent: floating-point equality.

Walk through it live: print the actual computed value.

```
print(200.00 * 0.07)
```

Output:

```
14.000000000000002
```

Explain: “0.07 cannot be represented *exactly* in binary floating-point — the same limitation behind nearly every ‘why doesn’t my decimal math come out exactly right’ surprise in any programming language, not just Python. tax really is 14.000000000000002, not 14.0 — so tax == 14.0 is, correctly, False. This is not a bug in Python; it’s an inherent property of how computers represent most decimal fractions in binary. **The fix: never compare floats with == when you expect a computed decimal result — use a tolerance-based comparison instead.**”

Show the fix:

```
import math

def test_tax_at_seven_percent():
    subtotal = 200.00
    tax = subtotal * 0.07
    assert math.isclose(tax, 14.0)
```

Bug 2 — test_over_limit_true and test_over_limit_false: chained comparison.

Walk through it live, printing the raw expression:

```
print(1500 > 1000 is True)
```

Output:

```
False
```

Explain, slowly, since this is genuinely one of the most surprising things in Python for a beginner: “This looks like it should ask ‘is 1500 > 1000 equal to True’ — but Python doesn’t read it that way. Python supports **chained comparisons**: a > b is c is actually evaluated as a > b

and `b is c` — comparing `b` on *both* sides at once, not evaluating `a > b` first and then comparing *that result* against `c`. So `1500 > 1000 is True` really means `1500 > 1000` and `1000 is True` — and `1000 is True` is `False` (an integer is never identical to the boolean `True`, even though `1000` is ‘truthy’ in a boolean context — `is` checks genuine identity, not truthiness). `True` and `False` is `False` — which is exactly the failing result we saw.” Point out, if visible in your terminal: **Python itself often prints a SyntaxWarning: “is” with ‘int’ literal. Did you mean “==”?** for this exact pattern — worth reading aloud, since Python is quite literally trying to warn about this specific mistake.

Show the fix — parentheses force the intended grouping:

```
def test_over_limit_true():
    assert (1500 > 1000) is True

def test_over_limit_false():
    assert (500 > 1000) is False
```

Even better, mention explicitly as the more idiomatic fix: “In practice, you’d almost never write `assert (x > y) is True` at all — `assert x > y` alone already asserts the condition is truthy, which is all a test needs. Comparing explicitly against `is True` is really only useful when you specifically need to distinguish a real `True` from some other truthy value — rare in practice. The parenthesized version above is shown because it directly fixes the *given* code with the smallest possible change, but the cleaner rewrite is worth mentioning.”

Re-run with all fixes applied. Verified output:

```
tests/test_week5.py::test_tax_at_seven_percent PASSED
tests/test_week5.py::test_over_limit_true PASSED
tests/test_week5.py::test_over_limit_false PASSED
tests/test_week5.py::test_type_of_string PASSED
tests/test_week5.py::test_type_conversion PASSED
5 passed
```

Common student mistakes to watch for:

- Assuming a failing test always means *their* code is wrong, rather than considering the test itself might be flawed — this is a genuinely valuable, humbling lesson from today specifically: tests are code too, and code can have bugs, including tests written by an instructor or a lab page. Model appropriate skepticism, not blind trust, toward any code — including this guide’s own corrections, worth saying explicitly and a little wryly.
- “Fixing” the floating-point issue by rounding `tax` to force it to equal `14.0` exactly (e.g., `tax = round(subtotal * 0.07, 2)` then `assert tax == 14.0`) — this actually works for this specific case, and is worth accepting as a valid alternative fix if a student proposes it, but flag explicitly that `math.isclose()` is the more general, robust pattern that doesn’t depend on the specific numbers involved happening to round cleanly.

Check for understanding: “Would `assert 1500 > 1000 == True` (using `==` instead of `is`) have the same chained-comparison problem?” (Yes — chaining applies to *any* comparison operators strung together this way, not just `is` specifically; `1500 > 1000 == True` would evaluate as `1500 > 1000` and `1000 == True`, and `1000 == True` is *also* `False` in Python, since `True` only equals `1`, not `1000`. Getting a student to generalize the lesson beyond the one specific operator (`is`) shown confirms the concept — not just the fix — actually landed.)

6.6 Part 5 — Submission Ritual and Push (1:03–1:11, 8 min)

Teaching goal: Execute Week 4’s full ritual on this week’s genuinely new content — the first time it’s run without step-by-step guidance, on a new project.

Say to the class:

“Same five steps as Week 4, on today’s work. I want to see hands move, not just watch me type.”

Live-code this:

```
ruff format .
ruff check .
pytest
git status
git add .
git commit -m 'lab 5: variables types operators'
git push
```

Line-by-line explanation: Identical to Week 4’s ritual — no new syntax here, purely repetition, which is the actual point: say explicitly, “this should feel *faster* and more automatic than it did three weeks ago — that’s the habit forming correctly.”

One new note specific to this module: since `module04_programming/` is a fresh project folder, this is its **first-ever** commit — if it also needs its own new GitHub remote (rather than reusing Week 4’s `ism3232-module02` repo), that’s a `git remote add origin ...` step identical to Week 4 Part 4, worth having ready to demonstrate again briefly if your course’s structure calls for a separate repo per module.

Verified — ritual complete when, per the same three criteria as Week 4:

- `ruff check` returns no errors
- `pytest` returns all 5 passed (after Part 4’s fixes)
- `git status`, after the push, says “nothing to commit”

Common student mistakes to watch for:

- Pushing **before** fixing Part 4’s two bugs — say explicitly, this week specifically: **do not commit and push a test file with known failing tests** — the whole point of today’s diagnostic work is to arrive at Part 5 with a genuinely green suite, not a red one pushed out of time pressure.
- Forgetting the commit message must include “lab 5” — the lab page’s own explicit grading requirement; a good final visual check before `git push`.

Check for understanding: “If `pytest` still showed failures at this point in the ritual, what should happen next — push anyway, or something else?” (Something else — go back and fix the failures)

first; the ritual's whole design is a sequence of gates, and a failing pytest step means the process shouldn't proceed to commit/push yet, exactly as established in Week 4.)

6.7 Stretch — Formatted Report Block (as time allows)

Frame as a quick close-out if the room reaches it:

```
approval_text = 'Manager required' if total > 1000 else 'Not required'
print('=== Purchase Request Summary ===')
print(f'Product:  {product_name:<12} Unit:  ${unit_price:.2f}')
print(f'Qty:      {quantity:<12} Total:  ${total:,.2f}')
print(f'Approval: {approval_text}')
```

Verified output (continuing the running Laptop/450.00/3 example):

```
=== Purchase Request Summary ===
Product:  Laptop          Unit:  $450.00
Qty:      3              Total:  $1,444.50
Approval: Manager required
```

One thing worth naming if you demo this: `approval_text = 'Manager required' if total > 1000 else 'Not required'` is a **conditional expression** (sometimes called a ternary) — a compact if/else that produces a value directly, in one line, rather than the multi-line if/else block Week 6 introduces formally. Worth flagging as a preview, not something to dwell on deeply today.

7 Wrap-Up (last ~4 minutes)

Review the submission checklist together:

- ☐ Git commit made, with a message including “lab 5”
- ☐ week5_lab.py contains all three parts (types, operators/f-strings, user input)
- ☐ tests/test_week5.py contains all five tests, corrected, all passing
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 6: “Today’s boolean checks (is_over_limit, requires_approval) just sat there as printed values. Next week, you finally branch on them — if/elif/else — plus loops and dictionaries, processing more than one purchase request at a time instead of just one.”

8 Appendix A — Full Answer Key (week5_lab.py + tests/test_week5.py)

```
# week5_lab.py
# Author: [Your Name]
# Business domain: Purchase request approval

# --- Part 1 ---
product_name = 'Laptop'
status       = 'Pending'
quantity     = 3
unit_price   = 450.00
is_over_limit = unit_price * quantity > 1000

print(type(product_name), type(quantity), type(unit_price), type(is_over_limit))

# --- Part 2 ---
subtotal      = unit_price * quantity
tax           = subtotal * 0.07
total         = subtotal + tax
requires_approval = total > 1000

print('=== Purchase Request Summary ===')
print(f'Product: {product_name}')
print(f'Qty: {quantity}')
print(f'Subtotal: ${subtotal:.2f}')
print(f'Tax: ${tax:.2f}')
print(f'Total: ${total:.2f}')
print(f'Requires approval: {requires_approval}')
```

```
# --- Part 3 ---
user_qty = int(input('Enter a new quantity: '))
new_total = unit_price * user_qty * 1.07
print(f'New total for {user_qty} units: ${new_total:.2f}')
print(f'Requires approval: {new_total > 1000}')
```

tests/test_week5.py — CORRECTED (see Part 4 for why the provided version fails):

```
# tests/test_week5.py
import math

def test_tax_at_seven_percent():
    subtotal = 200.00
    tax = subtotal * 0.07
    assert math.isclose(tax, 14.0)

def test_over_limit_true():
    assert (1500 > 1000) is True

def test_over_limit_false():
    assert (500 > 1000) is False

def test_type_of_string():
    name = 'ISM3232'
    assert type(name) == str

def test_type_conversion():
    s = '42'
    assert int(s) == 42
```

Stretch (formatted report block):

```
approval_text = 'Manager required' if total > 1000 else 'Not required'
print('=== Purchase Request Summary ===')
print(f'Product: {product_name:<12} Unit: ${unit_price:.2f}')
print(f'Qty: {quantity:<12} Total: ${total:,.2f}')
print(f'Approval: {approval_text}')
```

9 Appendix B — Extra Practice (only if the class finishes early)

Five required parts fill the full class period at a normal pace, especially given Part 4's diagnostic depth. If a section moves unusually fast:

Extra — a third precision trap. Have students predict, then verify: $0.1 + 0.2 == 0.3$ (the single most famous floating-point surprise in any language — verified: `False`, since $0.1 + 0.2$ actually evaluates to 0.30000000000000004). A good, quick, second real-world instance of Part 4's Bug 1, in a form students are likely to encounter again independently.

Extra — a second business scenario, from scratch. Have students build a second, independent version of Part 1–2's script for a different business domain (inventory restock, expense reimbursement, event registration) — all four data types, at least two operators, one f-string summary — good rehearsal of the whole pattern on content they design themselves rather than the provided template.